

POINTERS

Pointers

□ Pointers

- Powerful
- Simulate pass-by-reference
- Close relationship with arrays and strings

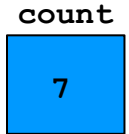
Pointers

- It provides a way of accessing a variable without referring to its name.
- The mechanism used for this is the *address* of the variable.

Pointer Variable Declarations and Initialization

□ Pointer variables

- Contain memory addresses as values
- Normally, variable contains specific value (direct reference)
- Pointers contain address of variable that has specific value (indirect reference)



□ Indirection

- Referencing value through pointer

□ Pointer declarations

- ***** indicates variable is pointer

```
int *myPtr;
```

declares pointer to **int**, pointer of type **int ***

- Multiple pointers require multiple asterisks

```
int *myPtr1, *myPtr2;
```

Pointer Variable Declarations and Initialization

- Can declare pointers to any data type
- Pointer initialization
 - Initialized to **0**, **NULL**, or address
 - **0** or **NULL** points to nothing

Address of variable

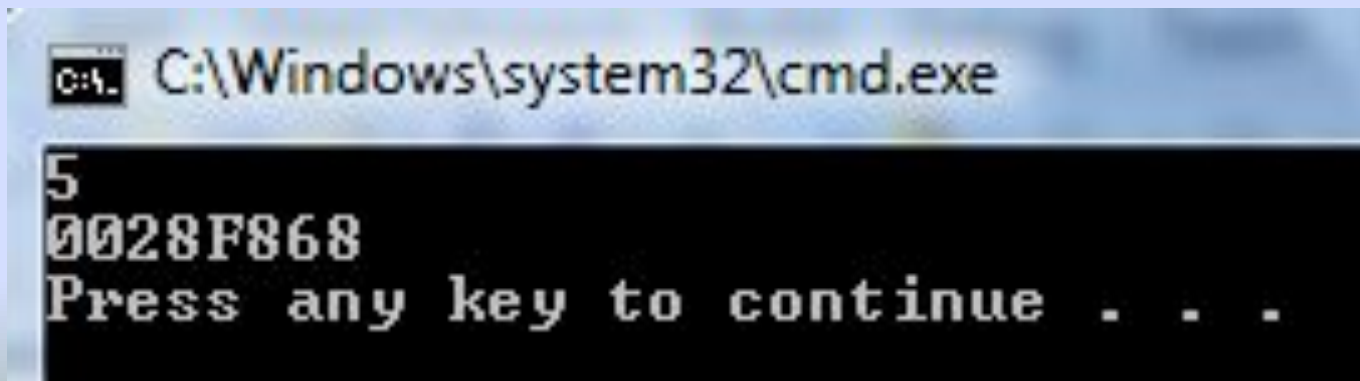
- Each variable occupies some bytes in memory (according to its size)
- Each byte of memory has a unique address so that it can be accessed (just like the address of our homes)
- Variable names are actually titles given to these addresses
 - Such as the 'Wapda house'
- When we use a variable, the compiler fetches the value placed at its address or writes a new value to that address

Address of variable

- How can we find the address of that variable?
- The “&” (Ampersand) operator returns the address of a variable

```
cout << a << endl;
```

```
cout << &a << endl;
```



A screenshot of a Windows command prompt window. The title bar shows the path 'C:\Windows\system32\cmd.exe'. The command prompt displays the output of a program: the number '5' on the first line, the memory address '0028F868' on the second line, and the text 'Press any key to continue . . .' on the third line.

```
C:\Windows\system32\cmd.exe  
5  
0028F868  
Press any key to continue . . .
```

Pointers

- Pointers are just variables storing numbers –but those numbers are memory addresses
- A pointer that stores the address of some variable *x* is said to *point to x*.
- A pointer is declared by putting a star (or '*') before the variable name.
- To access the **value** of *x*, we'd need to *dereference* the pointer.

Motivation for using Pointers

- To return more than one value from a function.
- To pass arrays and strings more conveniently from one function to another.
- To manipulate arrays more easily by moving pointers to them.
- Obtaining memory from the system
- In more advanced programming, such as
 - To create complex data structures, such as linked lists and binary trees
 - For dynamic memory allocation and de-allocation

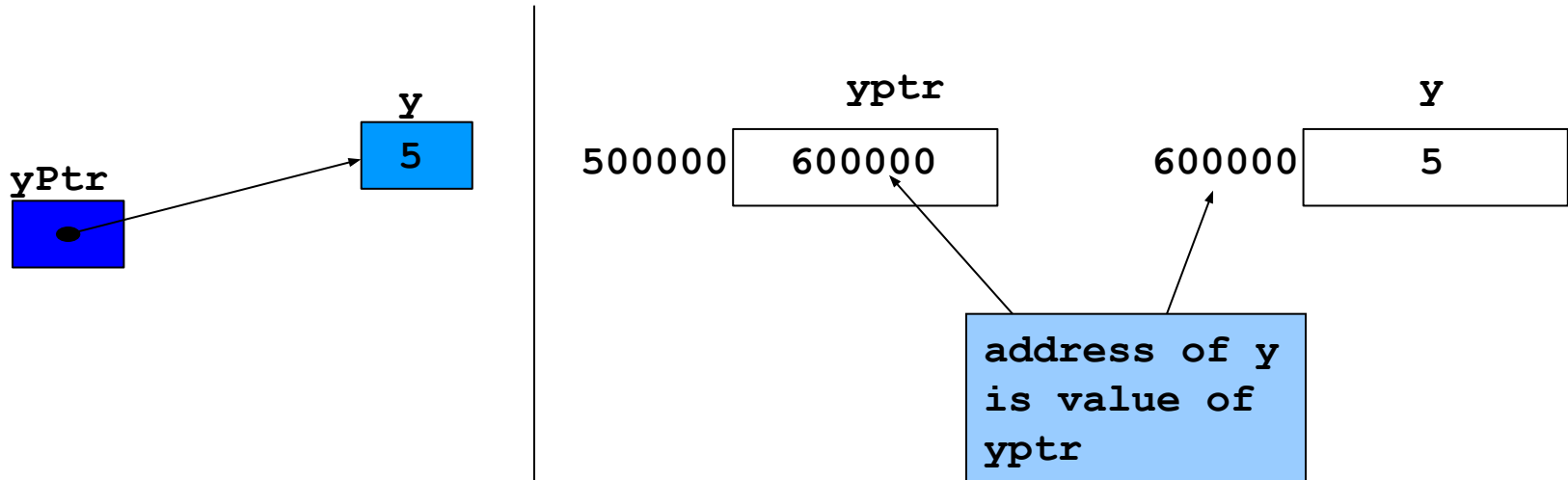
Pointer Operators

□ & (address operator)

- Returns memory address of its operand
- Example

```
int y = 5;  
int *yPtr;  
yPtr = &y;    // yPtr gets address of y
```

- **yPtr** “points to” **y**



Pointer Operators

- ***** (indirection/dereferencing operator)
 - Returns synonym for object its pointer operand points to
 - ***yPtr** returns **y** (because **yPtr** points to **y**).

```
*yPtr = 9;           // assigns 9 to y
```

- ***** and **&** are inverses of each other

```

1 // Fig. 5.4: fig05_04.cpp
2 // Using the & and * operators.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 int main()
9 {
10     int a;        // a is an integer
11     int *aPtr;    // aPtr is a pointer to an integer
12
13     a = 7;
14     aPtr = &a;    // aPtr assigned address of a
15
16     cout << "The address of a is " << &a
17           << "\nThe value of aPtr is " << aPtr;
18
19     cout << "\n\nThe value of a is " << a
20           << "\nThe value of *aPtr is " << *aPtr;
21
22     cout << "\n\nShowing that * and & are inverses of "
23           << "each other.\n&*aPtr = " << &*aPtr
24           << "\n*&aPtr = " << *&aPtr << endl;
25

```

* and & are inverses
of each other

```
26     return 0;  // indicates successful termination
27
28 } // end main
```

The address of a is 0012FED4

The value of aPtr is 0012FED4

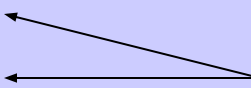
The value of a is 7

The value of *aPtr is 7

Showing that * and & are inverses of each other.

&*aPtr = 0012FED4

*&aPtr = 0012FED4



*** and & are inverses; same
result when both applied to
aPtr**

Calling Functions by Reference

- 3 ways to pass arguments to function
 - Pass-by-value
 - Pass-by-reference with reference arguments
 - Pass-by-reference with pointer arguments
- `return` can return one value from function
- Arguments passed to function using reference arguments
 - Modify original values of arguments
 - More than one value “returned”

Calling Functions by Reference

- Pass-by-reference with pointer arguments
 - Simulate pass-by-reference
 - Use pointers and indirection operator
 - Pass address of argument using **&** operator
 - Arrays not passed with **&** because array name already pointer
 - ***** operator used as alias/nickname for variable inside of function

```
1 // Fig. 5.6: fig05_06.cpp
2 // Cube a variable using pass-by-value.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 int cubeByValue( int ); // prototype
9
10 int main()
11 {
12     int number = 5;
13
14     cout << "The original value of number is "
15
16     // pass number by value to cubeByValue
17     number = cubeByValue( number );
18
19     cout << "\nThe new value of number is " << number << endl;
20
21     return 0; // indicates successful termination
22
23 } // end main
24
```

Pass number by value; result
returned by
cubeByValue

```
25 // calculate and return cube of integer argument
26 int cubeByValue( int n )
27 {
28     return n * n * n; // cube local variable
29
30 } // end function cubeByValue
```

cubeByValue receives
parameter passed-by-value

The original value of number is 5
The new value of number is 125

Cubes and **returns**
local variable **n**

```

1 // Fig. 5.7: fig05_07.cpp
2 // Cube a variable using pass-by-reference
3 // with a pointer argument.
4 #include <iostream>
5
6 using std::cout;
7 using std::endl;
8
9 void cubeByReference( int * ); // prototype
10
11 int main()
12 {
13     int number = 5;
14
15     cout << "The original value of number is " << number << endl;
16
17     // pass address of number to cubeByReference
18     cubeByReference( &number );
19
20     cout << "\nThe new value of number is " << number << endl;
21
22     return 0; // indicates successful termination
23
24 } // end main
25

```

Prototype indicates parameter
is pointer to **int**

Apply address operator **&** to
pass address of number to
cubeByReference

cubeByReference
modified variable
number

```
26 // calculate cube of *nPtr; modifies variable number in main
27 void cubeByReference( int *nPtr )
28 {
29     *nPtr = *nPtr * *nPtr * *nPtr; // cube
30
31 } // end function cubeByReference
```

cubeByReference
receives address of **int**
variable,
i.e., pointer to an **int**

The original value of number is 5
The new value of number is 125

Modify and access **int**
variable using indirection
operator *****

Pointers

- Declaration of pointer to an int

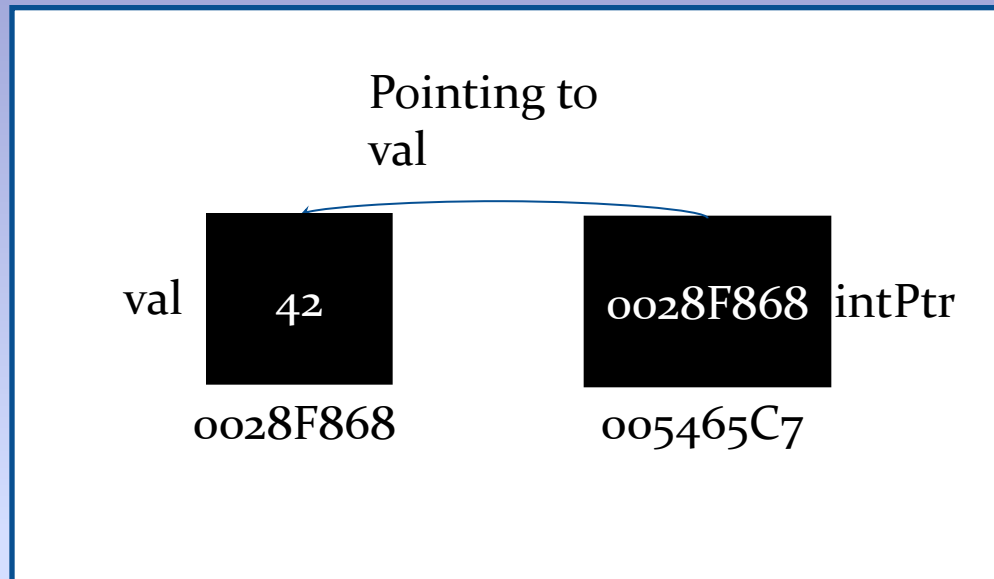
```
int* intPtr;
```

- Initializing a pointer

```
int val = 42;  
int* intPtr = &val;
```

- The ampersand (or '&') before val means "take the address of val".
- The pointer intPtr is assigned the address of val, which is another way of saying that intPtr now points to val.

Pointers



Pointers

- If we print out the value of `intPtr`, we'll get the address of `val`.
- To get the data that `intPtr` actually points at, we have to *dereference* `intPtr` with the *unary star operator* `*`

```
int val = 42;  
int *intPtr = &val;  
cout << "&val: " << &val << endl; //address of val  
cout << "intPtr: " << intPtr << endl; // ..again, address  
cout << "*intPtr: " << *intPtr << endl; // displays 42
```

- Note that the star (or `'*`') operator has many uses. When used in a binary context, it can mean multiplication. When used in a variable declaration, it means "this variable is a pointer". When used in an expression as a single-argument operator, it can mean "dereference". With so many uses, it's easy to get confused.

Pointers

- Since intPtr points to val, any changes that we make to val will also show up when we dereference intPtr:

```
int val = 42;
int* intPtr = &val;
cout << "val: " << val << endl; // displays 42
cout<<"*intPtr: "<<*intPtr<<endl;//displays 42
val = 999;
cout << "val: " << val << endl;// displays 999
cout << "*intPtr: " << *intPtr << endl; //
displays 999
```

Pointers

- You can declare a pointer to any type, not just int:

```
char ch= 'H';
```

```
char* chPtr = &ch;
```

```
cout << "ch: " << ch<< endl;
```

```
cout << "chPtr: " << chPtr << endl;
```

```
cout << "*chPtr: " << *chPtr << endl;
```

```
*chPtr = "!";
```

```
cout << "*chPtr: " << *chPtr << endl;
```

```
cout << "ch: " << ch<< endl;
```

Pointers

```
float val = 1.43;  
float* fltPtr = &val;  
cout << "val: " << val << endl;      // 1.43  
cout << "val: " << fltPtr << endl;    // address  
cout << "*fltPtr: " << *fltPtr << endl; // 1.43  
*fltPtr = 3.1416;  
cout << "*fltPtr: " << *fltPtr << endl; //3.1416  
cout << "val: " << val << endl;      // 3.1416
```

Pointers and **const**

- Pointers can be declared **const** in three ways:
 - The pointer itself can be declared **const**
 - The data the pointer points (“the pointee”) can be declared **const**
 - Both the pointer and the pointee can be declared **const**

Pointers and const

- **const** pointer, which means that once initialized it can't point at something else.

```
int val1 = 42;  
int * const intPtr = &val1;  
*intPtr = -1;           // okay  
int val2 = 999;  
intPtr = &val2;         // error!
```

Pointers and const

- **const** data, which means that the data that the pointer points to can't be changed

```
int val1 = 42;
```

```
const int * intPtr = &val1;
```

```
*intPtr = -1;           // error!
```

```
int val2 = 999;
```

```
intPtr = &val2;         // okay
```

Pointers and const

- Both of the above -- you can change neither the pointer nor the data it points to:

```
int val1 = 42;  
const int * const intPtr = &val1;  
*intPtr = -1;    // error!  
int val2 = 999;  
intPtr = &val2;  // error!
```

Pointers and const

- The variety of ways that you can use const can be confusing. One useful way to understand pointer declarations is to read them starting from the variable name reading from right to left .
 - `int * intPtr;` *//the variable intPtr is a pointer to an int.*
 - `const int * intPtr;` *//the variable intPtr is a pointer to an int //that is constant. Since the int is constant, we can't change it. We //can change intPtr, however.*
 - `int * const intPtr;` *//the variable intPtr is a constant pointer //to an int. Since intPtr is constant, we can't change it. However, we //can change the int.*
 - `const int * const intPtr;` *//the variable intPtr is a constant pointer to // an int that is constant. Everything is constant.*

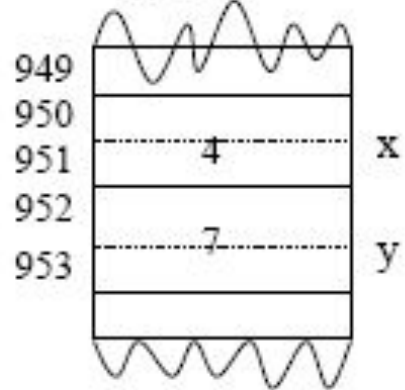
Example

```
int main ( )
{ int x=4, y=7;
  int *px, *py;
  px = &x; //int *px=&x;
  py = &y; //int *py=&y;
  cout << "x = " << x<<"y ="<<y;
  *px = *px + 10;
  *py = *py + 10;
  cout << "x = " << x<<"y ="<<y;
  return 0;
}
```

```
int x=4,y=7
```

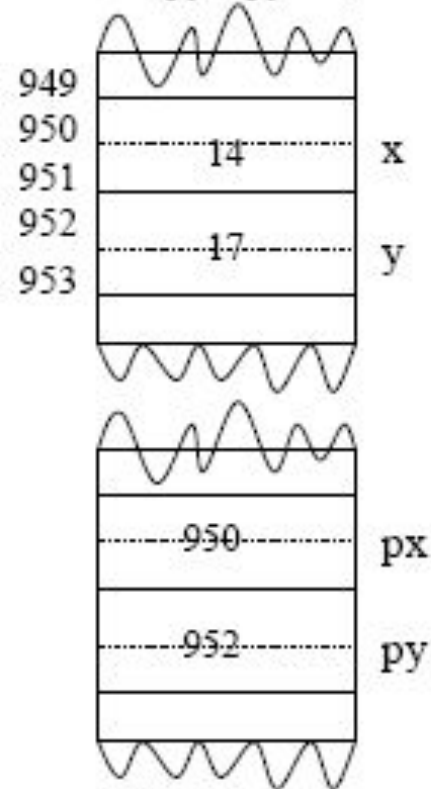
```
px=&x;
```

```
py=&y;
```



```
*px=*px+10;
```

```
*py=*py+10;
```



Operation of the addition using pointers

Discussion

```
int main (void)
{ int a;
  int *ptr1;
  char c;
  char *ptr2;
  int fred;
  ptr2=&c;      //ok. Char pointer init with add of a char
  ptr1=&fred;    //ok. int pointer init with the address of an int

  ptr1=&a ; //ok. int pointer init with the address of an
           int
```

Discussion

```
ptr2=&fred;    // NO. cannot assign the address of an int to a char
ptr2=&'#';     // NO. cannot take the address of an implicit constant
ptr2=&25;      // NO. cannot take the address of an implicit
               // constant
ptr2=&(a + 3);  // NO. cannot take the address of an expression
}
```

Discussion

```
int main()  
{ int c,a=10;  
  int *p1=&a;  
  c=*p1; //equivalent expression "c=a"  
  *p1=*p1**p1; //equivalent expression "a=a*a"  
  (*p1)++; //equivalent expression "a++"  
  c=*&a; //equivalent expression "c=a"  
}
```

Discussion

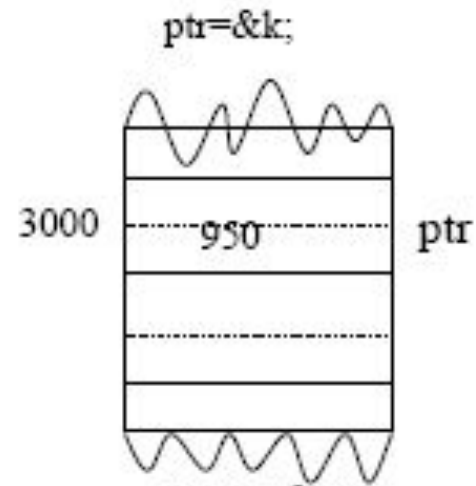
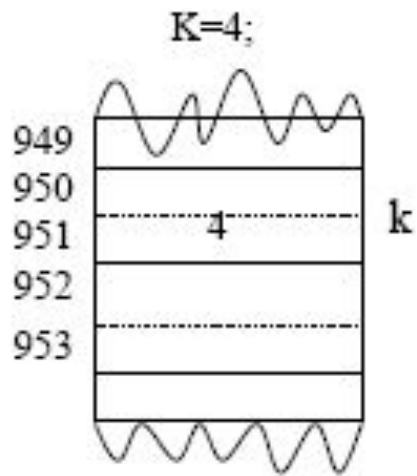
```
int a = 3;  
int b = 4;  
int *pointerToA = &a;  
int *pointerToB = &b;  
int *p = pointerToA;  
p = pointerToB;  
cout<< a << b << *p;    // Prints 344
```

Pointer to Pointer

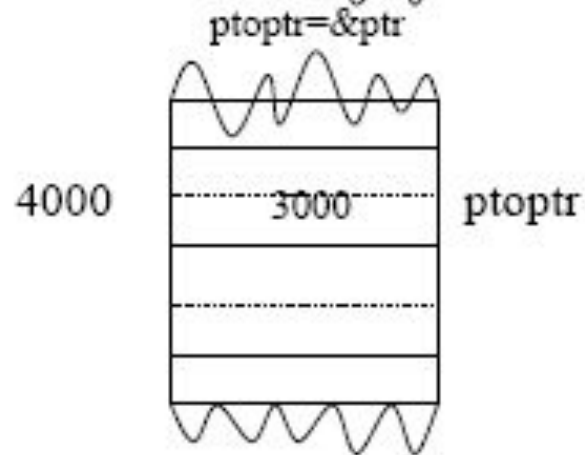
- `int k=4, *ptr, **ptoptr;`
- `ptr=&k`
- `ptoptr=&ptr;`

USE

- Creation of dynamically sizeable arrays
- Use of *new* and *delete*



- `int k=4,*ptr,**ptoptr;`
- `ptr=&k`
- `ptoptr=&ptr;`



```
int main ()
{
int x = 1;
int *px = &x;
int **ppx = &px;

cout << "x = " << x << endl;

*px = x+10;
cout << "x = " << x << endl;

**ppx = *px + x;
cout << "x = " << x << endl;

return 0;
}
```

```
int main ()  
{  
int x = 1;  
int *px = &x;  
int **ppx = &px;
```

```
cout
```

```
*px
```

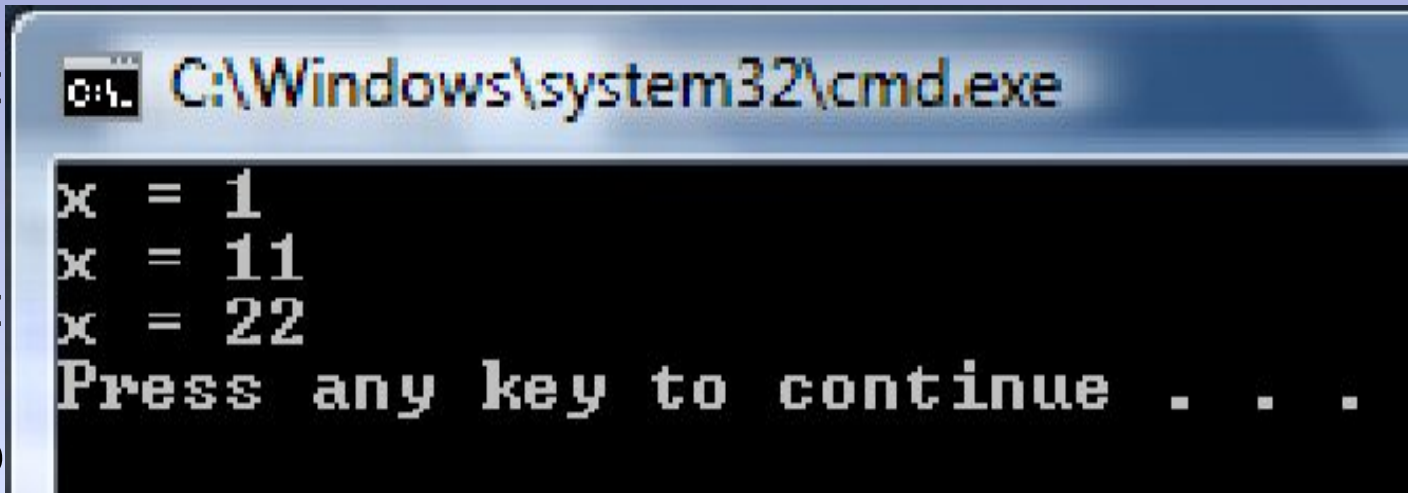
```
cout
```

```
**pp
```

```
cout << "x = " << x << endl;
```

```
return 0;
```

```
}
```



```
C:\Windows\system32\cmd.exe  
x = 1  
x = 11  
x = 22  
Press any key to continue . . .
```

Pointers and Arrays

- In c++, you can treat pointers and arrays very similarly:

```
int array[ ] = {3,1,4,1,5,9};  
int* arrayPtr = array;  
cout<<"array[0]: " <<array[0] << endl;//displays 3  
cout<<"arrayPtr[0]: "<<arrayPtr[0]<<endl;//displays 3  
arrayPtr++;  
cout<<"arrayPtr[0]: "<<arrayPtr[0]<<endl; // displays 1  
array++; // error: arrays are const
```

Pointers and Arrays

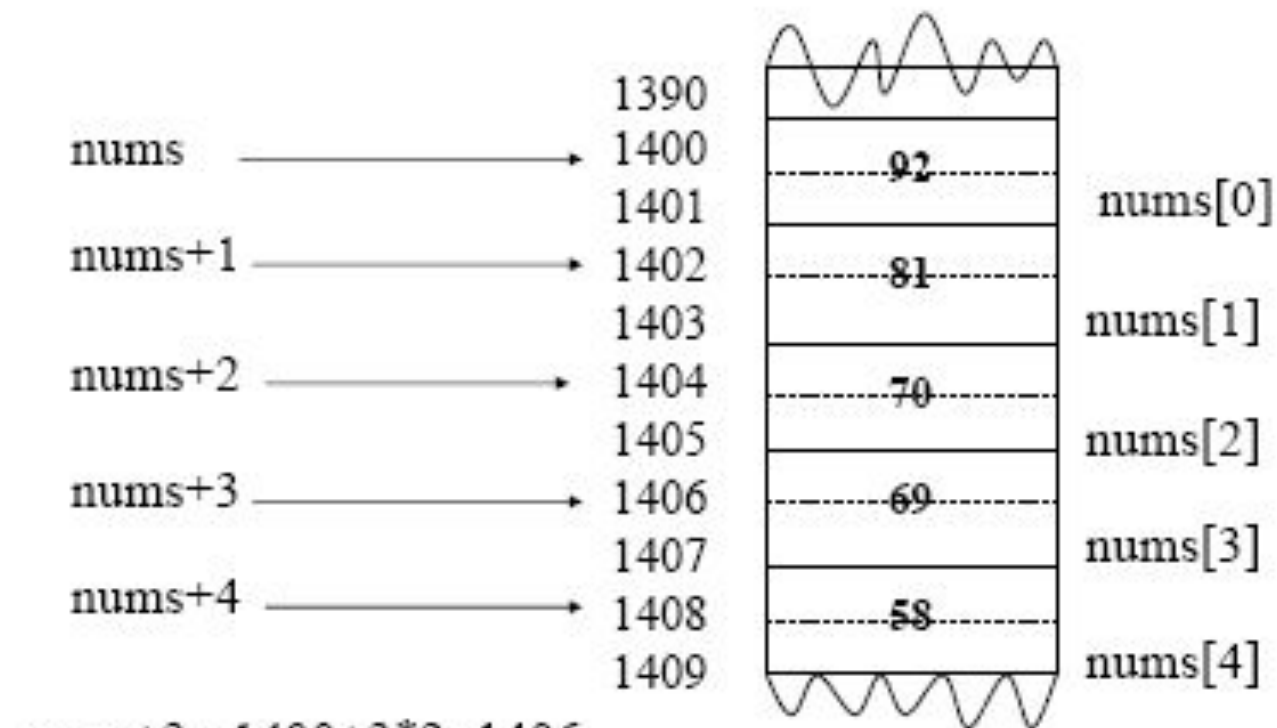
- Arrays and pointers may be used interchangeably, but you can change what pointers point at, whereas arrays will always "point" to the same thing.

Pointers and Arrays

// Using pointers to print out values from array

```
int main ( )  
{int nums [5] = {92,81,70,69,58};  
  int dex ;  
  for ( dex = 0; dex <5; dex++ )  
    cout << *(nums + dex)<<" ";  
}
```

● * (**nums + dex**) *is same as* **nums [dex]**.



$$\text{nums}+3 = 1400 + 3 * 2 = 1406$$

number of bytes
per integer.

Scalar value

Pointers and Functions

```
/* test function that returns two values */
void rets2 (int* , int* ); /*prototype*/
int main ( )
{
    int x, y; /* variables*/
    rets2 ( &x, &y ); /*get values from function*/
    cout <<"First is = "<< x<<"Second is = "<<y;
}

/* returns two numbers */
void rets2 ( int* px, int* py )
{
    *px= 3; /* set contents of px to 3 */
    *py = 5; /* set contents of py to 5 */
}
```